

Guava FDE — Take-Home Project

Thanks for making it to the final stage. This project is the core of your onsite: you'll build something real with Guava, then walk us through it in person during a project review.

We've designed this to look like the actual job. An FDE at Guava builds on top of our SDK, wires it into customer systems, and gets it into a state you'd put in front of real callers. That's what we're asking you to do here, in miniature.

A note on AI-assisted coding

Use AI coding tools if you want to — we do, and we encourage it. One thing to know about how we'll read your work: if you lean on AI, we hold what you deliver to the same bar as if you'd hand-written it. The tools make it cheap to produce a lot of code fast, so "it runs" isn't the interesting part — we're looking at whether you scoped well, caught what the tools got wrong, and made deliberate calls about what to keep, cut, and own yourself. You'll walk us through your build live and we'll dig into your decisions, so build something you can stand behind. (You'll also write a short AI-usage note — see *What to submit*.)

Getting started with your Guava sandbox

1. Sign in at <https://app.goguava.ai/dashboard/sign-in> and create your account.
2. Grab your two credentials from the dashboard sidebar:
 - [API Keys](#) → "Create API Key"
 - [Phone Numbers](#) → copy the number already provisioned on your account (no need to purchase one, it's set up for you)
3. Follow the SDK setup guide at <https://goguava.ai/docs/sdk-installation>. It walks you through installing the SDK, setting your `GUAVA_API_KEY` and `GUAVA_AGENT_NUMBER` environment variables, and running your first example agent.

Please reply with the email address you used to create your account so we can make sure your sandbox is set up correctly.

The docs are your starting point and primary reference. Start at <https://goguava.ai/docs> — the quickstart, SDK reference, and pattern guidance all live there. Build against what the docs describe; when a signature or behavior is in question, the SDK itself is ground truth. RAG is built

into the platform, so you don't need your own OpenAI key or a separate vector store for retrieval.

Time expectations

We expect this project to take roughly **8 hours**, some of which will likely go to getting set up with Guava and learning how to code in it. It's your first time in the platform, so budget for that.

Nobody gets something fully production-ready in 8 hours, and we're not asking you to. The bar below is a set of directions to push in, not a checklist to complete — we care as much about which gaps you chose to close first and which you consciously left open as about how far you got. If something runs long, cut scope and note in your README what you deprioritized and why.

Pick ONE of three projects

Choose the one that best fits how you like to work. For all three, *depth beats breadth*: an agent (or tool) that does two or three things well is far more compelling than one that does ten things shallowly.

First, pick a real company to build this for. Any real company that would plausibly use voice AI for your use case. Build the demo *for them* specifically: their domain, their callers, the things that would actually come up on their lines. A demo aimed at a concrete company is sharper than a generic one, and how you tailor it to them is part of what we're looking at.

Project A — Build an agent from scratch

Build a working Guava voice agent for a use case of your choosing, from scratch — your own design, using the SDK directly, working from the docs. This track is about authoring the agent yourself, not adapting an existing demo. You own the whole thing: the conversation design, the logic, the integration, and a story for how it would get deployed.

Must include:

- A working voice agent someone can actually call and complete a task with.
- At least 5 test scenarios with clear pass/fail criteria — including failure cases, not just the happy path. Pick the ones you think matter most and explain your choices.
- A short runbook for how you'd deploy this to a new customer/instance.

Project B — Build software on top of Guava

Build a small piece of software that sits on top of Guava, where the end user never sees Guava. They express what they want in plain domain terms and your software turns that into a running Guava agent.

The interface is up to you — it does **not** have to be a UI (but can be if appropriate). A defined input format (a JSON or YAML spec, a config file, a simple schema) is perfectly good, and sometimes a cleaner choice. What matters is that the input is expressed in the *problem domain* and hides Guava entirely, while your software compiles it into a working agent.

Must include:

- A working system: a domain-shaped input on one end, a running Guava agent on the other.
- In your README, be explicit about what the input format captures, what it deliberately leaves out, and how you'd extend it to a second use case.

Project C — Make an example more production-ready

The starter repo at <https://github.com/goguava-ai/guava-starter> has an [examples/](#) directory of simple demos organized by vertical. Pick one — or combine several that naturally belong together (for instance, several call types that a single real inbound line would need to handle and route between) — and move it meaningfully closer to something you'd put in front of real callers.

Two notes on the starter repo if you take this track:

- **Go straight to the [examples/](#) directory.** You can ignore the PRD-template workflow described in the README.
- **It's a real starter repo, not a polished tutorial.** You may hit rough edges — misused APIs, things that don't work quite as documented, examples that need fixing. That's intentional and realistic: a customer's codebase is never clean either. Finding your way around those, and deciding what's worth fixing, is part of the exercise — tell us how you did it in your walkthrough.

Must include:

- Decide what "more production-ready" means for your chosen use case and get it there (see the bar below).
- Keep the original example(s) unchanged alongside your version so we can diff.
- A short written description of each meaningful change: what you did, why, and what you considered and rejected.

The bar: what "production-minded" means (for all three)

Whichever track you pick, we're looking for something moving toward real callers, not a demo that only survives the happy path. You won't get all the way there in the time you have, and that's expected — treat the items below as the dimensions we care about and push each as far as your scope allows.

- **The conversation flows naturally.** Reasonable dialog, graceful recovery when it doesn't understand, no getting stuck in loops. You may not perfect this (it's genuinely hard), but we want to see you care about how the call feels to the person on the other end.
- **It behaves the way a competent human agent would** (within the natural limits of AI). Think about how a good human handling this call would actually behave — what they'd confirm, when they'd push back, when they'd hand off — and aim for that.
- **It integrates with a real API.** Your agent must call out to a real endpoint over the network and use the response in the conversation — book a slot, look up an order, check eligibility, whatever fits. A public API is fine; a local or mock server you stand up is also fine, so you never need real or sensitive data (fake data is expected and encouraged for anything regulated). We're looking for the integration, not the data — don't hardcode the answer into the prompt — and handle the call failing (timeouts, malformed responses, "not found") gracefully. How you handle things going wrong tells us more than the happy path.
- **It withstands what really comes up.** The objections, confusions, off-topic questions, and edge cases that would actually occur in that situation. Pick the ones that matter most for your use case and handle them.

A core idea in Guava is that you can enforce structure and control in deterministic code rather than relying on the model to always behave. As you build, consider what belongs in code around the model versus in the model's reasoning, and be ready to talk about the tradeoffs you made.

Optional bonus — a second modality. If (and only if) it genuinely serves your use case, you may add a second modality — an SMS confirmation, a text/voice handoff, a link sent mid-call, keypad input. Only if it serves the problem, not for its own sake. Good scope judgment is worth more to us than an extra feature.

What to submit

Share a **GitHub repo with us** containing:

- **Working code**, with **real commit history** — commit as you go. We want to see how you iterated, not one squashed commit at the end.
- A **README** covering: what you built, what you deliberately chose not to build and why, how someone would run it, and what you'd do with another 8 hours.

- An **AI-usage note** (a paragraph or two): how you used AI coding tools — what you delegated, where you overrode them, a place the tool got something wrong that you caught, and anywhere you chose to hand-code and why. We're interested in your judgment about it, not the volume.
-

The onsite

Plan for about **2 hours on site**. Of that, **30–45 minutes is the project review** where you walk us through what you built (details below). The rest is a couple of shorter, non-technical sessions with other members of the team.

The project review

Treat the demo like you're showing it to a customer — talk about decisions and outcomes, not line-by-line code. Come ready to cover:

Demo — show it working, and show it breaking

- **Happy paths:** the core experience working end to end.
- **Hard paths:** where it gets difficult — an integration failure, a confused or difficult caller, an objection, an edge case — and how your agent handles (or doesn't) it. We'd rather see you honestly show a rough edge than hide it.

Decisions — how and why you built it the way you did

- **Architecture:** how you structured it and why. What you put in code versus in the prompt, and how you decided. If you worked around anything broken or surprising in the starter repo, tell us how.
- **AI-assisted coding:** how you drove the tools, where they helped, where they got something wrong and how you caught it, and what you chose to own yourself.
- **Product strengths:** where Guava genuinely helped.
- **Product weaknesses:** a real, structural weakness you hit while building — not a cosmetic nitpick, but something a customer doing this for real would run into. Frame it for a product manager deciding whether to fix it: what it impacts and for whom, why it matters, and how urgent it is.

Be ready for follow-up questions throughout, and to talk about how you'd extend and improve what you built.

Questions

Reach out any time before the onsite. Knowing when to ask a clarifying question versus when to make a reasonable call and move on is part of the job — so use your judgment, and don't hesitate if you're genuinely stuck.

Our docs may be missing information that you need; it's entirely appropriate to surface that. But, do try to discover what you can on your own through inspection of the SDK and/or experimentation.